

Decorator, Chain of Responsibility & MVC Patterns in Spring Boot Microservices

Decorator, Chain of Responsibility & MVC Patterns in Java Spring Boot Microservices (0-7 Years Guide)

PART 1 — DECORATOR PATTERN

1. What It Is

Decorator lets you **attach new behavior to an object dynamically** by wrapping it in another object that implements the same interface, without modifying the original class or using subclassing for every combination of behavior.

Why it matters: avoids a combinatorial explosion of subclasses when you need to mix and match behaviors (e.g., compressed + encrypted + logged stream) — you just stack wrappers.

2. Classic Java Implementation

```
public interface DataSource {
    String read();
    void write(String data);
}

public class FileDataSource implements DataSource {
    public String read() { return "raw file data"; }
    public void write(String data) { System.out.println("Writing to
file: " + data); }
}

// Decorator – wraps a DataSource and adds behavior, same interface
public class EncryptedDataSource implements DataSource {
    private final DataSource wrapped;

    public EncryptedDataSource(DataSource wrapped) { this.wrapped =
wrapped; }

    public String read() { return decrypt(wrapped.read()); }
    public void write(String data) { wrapped.write(encrypt(data)); }

    private String encrypt(String data) { return "ENC(" + data +
");" }
    private String decrypt(String data) { return
data.replace("ENC(", "").replace(")", ""); }
}

// Another decorator, stackable on top
```

```

        public class CompressedDataSource implements DataSource {
            private final DataSource wrapped;
            public CompressedDataSource(DataSource wrapped) { this.wrapped =
wrapped; }
            public String read() { return decompress(wrapped.read()); }
            public void write(String data) { wrapped.write(compress(data)); }
        }

        private String compress(String d) { return "ZIP(" + d + ")"; }
        private String decompress(String d) { return d.replace("ZIP(",
"".replace(")", ""); }
    }

```

Usage — behaviors stack in any combination without new subclasses:

```

DataSource source = new CompressedDataSource(new
EncryptedDataSource(new FileDataSource()));
source.write("hello"); // encrypted, then compressed, then written

```

3. HttpServletRequestWrapper — Real-World Decorator in Spring/Servlet

HttpServletRequest is an interface. You can't (and shouldn't) change the servlet container's implementation, but you often need to **modify or enrich request data** before it reaches your controller — e.g., re-reading a request body multiple times, adding a custom header, masking sensitive parameters. The standard approach is wrapping it with HttpServletRequestWrapper, which is Java's Decorator implementation for HttpServletRequest.

```

        public class CachedBodyHttpServletRequest extends
HttpServletRequestWrapper {
            private final byte[] cachedBody;

            public CachedBodyHttpServletRequest(HttpServletRequest request)
throws IOException {
                super(request); // wraps the original request
                InputStream requestInputStream = request.getInputStream();
                this.cachedBody =
StreamUtils.copyToByteArray(requestInputStream);
            }

            @Override
            public ServletInputStream getInputStream() {
                // returns a fresh stream each time – solves the "body can
only be read once" problem
                return new CachedBodyServletInputStream(cachedBody);
            }
        }

```

Used in a filter so downstream code (logging filters, controllers) can read the request body more than once:

```

@Component
public class CachingRequestBodyFilter extends OncePerRequestFilter {
    @Override
    protected void doFilterInternal(HttpServletRequest request,
HttpServletRequestResponse response,
                                FilterChain filterChain) throws
ServletException, IOException {
        CachedBodyHttpServletRequest wrappedRequest = new
CachedBodyHttpServletRequest(request);
        filterChain.doFilter(wrappedRequest, response); // pass the
DECORATED request onward
    }
}

```

Similarly, `HttpServletResponseWrapper` decorates the response — commonly used to capture the response body for logging before it's sent to the client.

4. Where Decorator Shows Up in Microservices

- Request/response body caching/logging filters (as above) — very common for audit logging middleware
- Adding correlation IDs / masking sensitive fields in logs by wrapping request/response objects
- Wrapping `InputStream/OutputStream` for compression, encryption, or checksum validation on file uploads
- Wrapping a `DataSource` (JDBC) to add query logging or multi-tenancy schema switching without touching the real connection pool implementation

```
public class LoggingDataSource implements DataSource {
    private final DataSource delegate;
    public Connection getConnection() throws SQLException {
        System.out.println("Connection requested at " +
Instant.now());
        return delegate.getConnection();
    }
    // other DataSource methods delegate directly
}
```

5. Decorator vs Proxy vs Adapter (Frequently Asked Together)

Pattern	Interface	Intent
Decorator	Same interface as wrapped object	Add new behavior/responsibility
Proxy	Same interface as wrapped object	Control access (security, caching, lazy loading) — structurally identical to Decorator, but intent differs
Adapter	Different interface than wrapped object	Convert an incompatible interface into a usable one

This structural similarity between Proxy and Decorator is one of the most common “explain the difference” interview questions — the code often looks nearly identical; the distinguishing factor is *intent* (access control vs behavior enhancement).

6. Decorator — By Experience Level

0-2 Years

Expected: Know what Decorator solves (adding behavior without subclassing), recognize

`HttpServletRequestWrapper/HttpServletResponseWrapper` as decorators.

Common questions: What is the Decorator pattern? Why not just use inheritance to add behavior?

2-4 Years

Expected: Write a request/response body-caching filter using `HttpServletRequestWrapper`; explain why request bodies can normally only be read once; stack multiple decorators cleanly. **Common questions:** How would you log the request/response body in a filter without breaking the controller's ability to read it? Difference between Decorator and Proxy?

4-7 Years

Expected: Design layered decorators for cross-cutting request/response processing (masking PII in logs, checksum validation, multi-tenant routing) across a whole gateway/filter chain; understand performance cost of wrapping streams/requests at scale (buffering large request bodies in memory); combine Decorator with Chain of Responsibility (filters chaining wrapped requests) cleanly. **Common questions:** Design a request-body-logging solution for an API gateway handling large file uploads — what are the memory/performance trade-offs of the `HttpServletRequestWrapper` approach? How would you mask sensitive fields (card numbers, passwords) in logged request bodies without modifying every controller?

PART 2 — CHAIN OF RESPONSIBILITY PATTERN

1. What It Is

Chain of Responsibility passes a request along a **chain of handlers**, where each handler decides either to process the request, pass it to the next handler, or (in some designs) do both — the sender doesn't need to know which handler will ultimately process it, or how many handlers exist.

Why it matters: decouples senders from receivers; lets you add/remove/reorder processing steps (auth, logging, validation) without changing the core request-handling code.

2. Classic Java Implementation

```
public abstract class Handler {
    protected Handler next;

    public Handler setNext(Handler next) { this.next = next; return
next; }

    public abstract void handle(Request request);
}

public class AuthHandler extends Handler {
    public void handle(Request request) {
        if (!request.isValidToken()) {
            throw new SecurityException("Unauthorized");
        }
        System.out.println("Auth passed");
        if (next != null) next.handle(request); // pass along the
chain
    }
}

public class LoggingHandler extends Handler {
    public void handle(Request request) {
```

```

        System.out.println("Logging request: " + request);
        if (next != null) next.handle(request);
    }
}

public class BusinessLogicHandler extends Handler {
    public void handle(Request request) {
        System.out.println("Processing business logic");
        // end of chain – no next.handle() call
    }
}

```

Usage:

```

Handler chain = new AuthHandler();
chain.setNext(new LoggingHandler()).setNext(new
BusinessLogicHandler());
chain.handle(new Request());

```

3. Servlet Filters — Real Chain of Responsibility in the Servlet Spec

Every incoming HTTP request in a Java web app passes through a **chain of Filter objects** before reaching your DispatcherServlet/controller. Each filter decides whether to pass the request further down the chain.

```

@Component
public class CorrelationIdFilter extends OncePerRequestFilter {
    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain filterChain) throws
ServletException, IOException {
        String correlationId = request.getHeader("X-Correlation-
Id");
        if (correlationId == null) correlationId =
UUID.randomUUID().toString();
        MDC.put("correlationId", correlationId);

        try {
            filterChain.doFilter(request, response); // pass to the
NEXT filter/servlet in the chain
        } finally {
            MDC.clear(); // always cleanup, even if downstream
throws
        }
    }
}

```

If this filter simply doesn't call `filterChain.doFilter(...)`, the request stops here — never reaching the controller. That's the essence of Chain of Responsibility: each handler controls whether the request continues.

4. Spring Security Filter Chain — The Most Prominent Example

Spring Security is built entirely around this pattern — a request passes through an ordered chain of security filters, each handling one concern:

```

Request → CorsFilter → CsrfFilter → LogoutFilter →
UsernamePasswordAuthenticationFilter
        → BasicAuthenticationFilter → ExceptionTranslationFilter →
FilterSecurityInterceptor → Controller

```

Configuring it declaratively:

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws
Exception {
        http
            .csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/public/**").permitAll()
                .anyRequest().authenticated()
            )
            .addFilterBefore(new JwtAuthFilter(),
UsernamePasswordAuthenticationFilter.class);
        return http.build();
    }
}
```

Each filter in this chain: - Can **short-circuit** the chain (e.g., reject with 401 if the JWT is invalid, never reaching the controller) - Can **enrich** the SecurityContext (e.g., set the authenticated user) before passing control forward - Doesn't know or care what filters come before/after it — pure decoupling

Custom filter example:

```
public class JwtAuthFilter extends OncePerRequestFilter {
    @Override
    protected void doFilterInternal(HttpServletRequest request,
HttpServletResponse response,
                                FilterChain filterChain) throws
ServletException, IOException {
        String token = extractToken(request);
        if (token != null && jwtValidator.isValid(token)) {
            Authentication auth =
jwtValidator.getAuthentication(token);

            SecurityContextHolder.getContext().setAuthentication(auth);
        }
        filterChain.doFilter(request, response); // continue the
chain either way; downstream decides access
    }
}
```

5. Where Chain of Responsibility Is Used in Microservices

- Servlet filters: CORS, correlation ID injection, request/response logging, rate limiting
- Spring Security filter chain: authentication, authorization, CSRF protection
- API Gateway filter chains (Spring Cloud Gateway GlobalFilters — pre/post filters per route)
- Validation pipelines: chain of validators, each checking one rule, short-circuiting on first failure
- Resilience4j's decorator-based composition (circuit breaker → retry → rate limiter → bulkhead — each wraps the next)

```
@Component
public class RequestSizeFilter implements GlobalFilter, Ordered {
    @Override
    public Mono<Void> filter(ServerWebExchange exchange,
GatewayFilterChain chain) {
        if (exchange.getRequest().getHeaders().getContentTypeLength() >
```

```

MAX_SIZE) {

exchange.getResponse().setStatusCode(HttpStatus.PAYLOAD_TOO_LARGE);
        return exchange.getResponse().setComplete(); // chain
stops here
    }
    return chain.filter(exchange); // continue to next filter in
Spring Cloud Gateway's chain
}
@Override
public int getOrder() { return -1; } // controls position in the
chain
}

```

6. Chain of Responsibility — By Experience Level

0-2 Years

Expected: Know that servlet filters run before controllers, basic idea of a filter chain. **Common questions:** What is a Servlet Filter? What's the difference between a Filter and an Interceptor?

2-4 Years

Expected: Write custom filters (OncePerRequestFilter), understand filter ordering (@Order, addFilterBefore/addFilterAfter), configure a basic Spring Security filter chain. **Common questions:** How would you add a correlation ID to every request for tracing? Explain the difference between a Filter (Servlet spec) and a Spring Interceptor (HandlerInterceptor) — when would you use each?

4-7 Years

Expected: Design a full custom authentication/authorization filter chain (JWT validation, role-based access, multi-tenant routing) with correct filter ordering; design API Gateway-level filter chains for cross-cutting concerns across all microservices (rate limiting, request size limits, auth) rather than duplicating logic per service; understand how filter chain short-circuiting affects observability/tracing (ensuring correlation IDs propagate even on early rejection); combine Chain of Responsibility with Resilience4j's fault-tolerance decorators.

Common questions: Design the filter chain for a microservice that needs CORS, JWT auth, rate limiting, and request logging — what order do they go in, and why? How do you ensure a request rejected early in the filter chain (e.g., failed auth) still gets logged/traced properly? How does Spring Cloud Gateway's filter chain differ architecturally from a traditional Servlet filter chain (blocking vs reactive)?

PART 3 — MVC (SPRING WEB'S CORE ARCHITECTURE)

1. What It Is

MVC (Model-View-Controller) separates an application into three responsibilities: - **Model** — the data/business state - **View** — how that data is presented - **Controller** — receives input, coordinates between Model and View

In a Spring Boot **REST API microservice**, there's typically no server-rendered View — the "View" is effectively the JSON response body, so MVC here mostly means Controller + Model (DTOs) with the "View" step reduced to serialization.

2. The Core Flow — DispatcherServlet

Every request into a Spring MVC app funnels through one central component, the **DispatcherServlet**, which acts as the Front Controller:

```
Client Request
↓
DispatcherServlet (front controller – single entry point)
↓
HandlerMapping (figures out which @Controller method should
handle this URL)
↓
HandlerAdapter (Adapter pattern! – invokes the handler method
uniformly)
↓
@Controller method (your code – Model built here)
↓
HttpMessageConverter (serializes the returned object to JSON – the
"View" step in a REST API)
↓
Client Response
```

```
@RestController
@RequestMapping("/orders")
public class OrderController {
    private final OrderService orderService;

    public OrderController(OrderService orderService) {
        this.orderService = orderService;
    }

    @GetMapping("/{id}")
    public ResponseEntity<OrderResponseDto> getOrder(@PathVariable
Long id) {
        OrderResponseDto order = orderService.getOrder(id); // Model
        return ResponseEntity.ok(order); // "View" = JSON
        // serialization via HttpMessageConverter
    }
}
```

@RestController = @Controller + @ResponseBody — meaning "serialize the return value directly into the response body" instead of resolving a server-side view template (JSP/Thymeleaf), which is what traditional MVC web apps do.

3. Traditional MVC (With an Actual View) vs REST-Style MVC

```
// Traditional MVC – returns a view NAME, resolved to an HTML
template
@Controller
public class OrderPageController {
    @GetMapping("/orders/{id}/view")
    public String viewOrder(@PathVariable Long id, Model model) {
        model.addAttribute("order", orderService.getOrder(id)); //
Model
        return "order-detail"; // View name – resolved to order-
detail.html (Thymeleaf)
    }
}

// REST-style MVC (what microservices actually use) – no server-
```

```

rendered view at all
@RestController
public class OrderApiController {
    @GetMapping("/api/orders/{id}")
    public OrderResponseDto getOrder(@PathVariable Long id) {
        return orderService.getOrder(id); // "View" = JSON, handled
by Jackson via HttpMessageConverter
    }
}

```

Microservices almost exclusively use the second style — the “View” layer is effectively delegated to whatever client (a frontend SPA, another microservice, a mobile app) consumes the JSON.

4. Key Building Blocks Inside Spring MVC

Component	Role
DispatcherServlet	Front controller — single entry point for all requests
HandlerMapping	Maps a URL + HTTP method to the right controller method
HandlerAdapter	Invokes the handler method (Adapter pattern applied internally)
HttpMessageConverter	Converts Java objects ↔ JSON/XML (Jackson by default)
HandlerInterceptor	Pre/post-processing around controller invocation (similar role to filters, but Spring-MVC-aware - has access to the handler method itself)
ExceptionHandlerExceptionHandlerResolver	Routes exceptions to @ExceptionHandler/@ControllerAdvice methods

```

@RestControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(OrderNotFoundException.class)
    public ResponseEntity<ApiErrorResponse>
handleNotFound(OrderNotFoundException ex) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(new ApiErrorResponse(404, ex.getMessage()));
    }
}

```

5. Filter vs Interceptor (Frequently Confused With Chain of Responsibility Section Above)

	Servlet Filter	HandlerInterceptor
Defined by	Servlet spec (framework-agnostic)	Spring MVC-specific
Runs	Before DispatcherServlet even starts routing	After DispatcherServlet has resolved the handler, but before/after the controller method runs
Access to handler method	No	Yes — can inspect @Controller method/annotations

Typical use	CORS, auth, correlation IDs, compression	Logging tied to specific controller methods, @RequiresPermission-style annotation checks
-------------	--	--

6. Where MVC Architecture Matters in Microservices

- Every REST microservice built with spring-boot-starter-web runs entirely on this MVC skeleton, whether or not you ever think about it directly
- Understanding HandlerInterceptor vs Filter matters when deciding where to put cross-cutting logic (raw HTTP-level vs handler-aware)
- @RestControllerAdvice + @ExceptionHandler is the standard way to centralize error response formatting across all controllers in a microservice
- Choosing between Spring MVC (Servlet-based, blocking) and Spring WebFlux (reactive, non-blocking, different core pipeline built on HandlerFunction/RouterFunction instead of @Controller) is a major architectural decision for high-throughput microservices

7. MVC — By Experience Level

0-2 Years

Expected: Build basic @RestController endpoints, understand @RequestMapping/@GetMapping/@PostMapping, know @RestController = @Controller + @ResponseBody. **Common questions:** What is MVC? What does @RestController do differently from @Controller? What is DispatcherServlet?

2-4 Years

Expected: Use @ControllerAdvice/@ExceptionHandler for centralized error handling, write HandlerInterceptors, understand HttpMessageConverter's role in serialization, differentiate Filter vs Interceptor. **Common questions:** How would you implement centralized exception handling across all your controllers? Difference between a Filter and a HandlerInterceptor — when would you use each? How does Spring know to convert your returned object into JSON?

4-7 Years

Expected: Explain the full DispatcherServlet request lifecycle in depth; make the Spring MVC vs Spring WebFlux architectural decision for a given microservice's throughput/concurrency needs; design a consistent, versioned API error-response contract across many microservices; understand how HandlerAdapter (Adapter pattern) lets DispatcherServlet uniformly invoke very different handler types (annotated controllers, functional endpoints, WebSocket handlers). **Common questions:** Walk through what happens internally between a request hitting your app and your @GetMapping method executing. When would you choose Spring WebFlux over Spring MVC for a microservice, and what changes architecturally? How would you design a consistent error-response format shared across 20 microservices maintained by different teams? Why is HandlerAdapter needed at all — why can't DispatcherServlet just call controller methods directly?

Quick Summary Table

Pattern	Core Purpose	Key Spring Example
Decorator	Add behavior to an object dynamically, same interface	HttpServletRequestWrapper, HttpServletResponseWrapper
Chain of Responsibility	Pass a request through an ordered chain of independent handlers	Servlet Filters, Spring Security filter chain, Spring Cloud Gateway filters
MVC	Separate request handling, data, and presentation	DispatcherServlet, @RestController, HttpMessageConverter

How They Combine in a Real Request (Senior-Level Integration Point)

"Trace a single POST /orders request through your entire microservice stack."

1. Request hits the **Chain of Responsibility** — Servlet filters run in order: CORS filter → correlation ID filter → Spring Security's JWT auth filter chain
2. A filter along the way may wrap the request using **Decorator** (HttpServletRequestWrapper) to cache the body for later logging
3. Once past all filters, the **MVC** framework takes over: DispatcherServlet → HandlerMapping → HandlerAdapter → your @RestController method
4. Your controller returns a DTO, serialized to JSON via HttpMessageConverter — completing the MVC cycle
5. If an exception occurs anywhere in step 3-4, @RestControllerAdvice (part of the MVC exception-resolution pipeline) formats a consistent error response

```
// Filter (Chain of Responsibility) wraps request (Decorator) before
MVC even starts
public class BodyCachingFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest request,
                                   HttpServletResponse response,
                                   FilterChain chain) throws
ServletException, IOException {
    CachedBodyHttpServletRequest wrapped = new
CachedBodyHttpServletRequest(request); // Decorator
    chain.doFilter(wrapped, response); // continues the Chain of
Responsibility into MVC's DispatcherServlet
    }
}
```